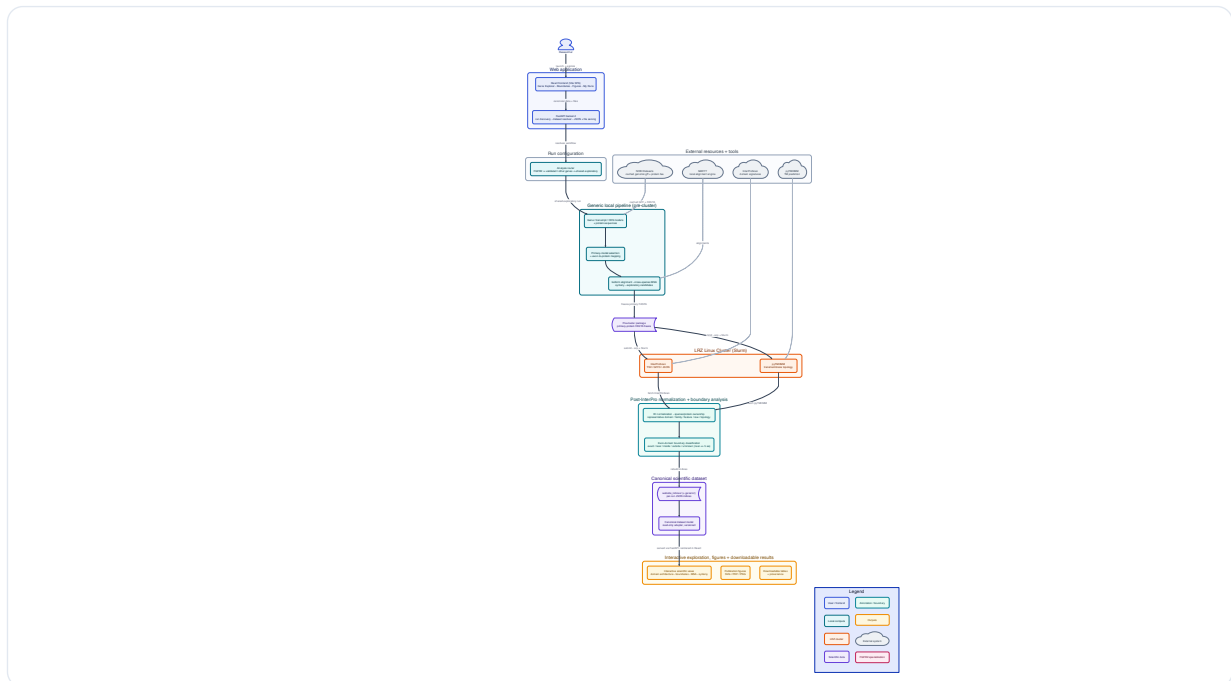

ExonDomainCompare

Architecture Guide

Annotation-aware comparative exon–protein analysis — one generic framework, FGFR2 as a validated specialization.



This guide is derived from the current ExonDomainCompare codebase. All diagrams are reproducible vector artifacts built from editable sources in docs/architecture/sources/ via python scripts/docs/build_architecture_atlas.py. Features marked *pending*, *optional* or *planned* are not yet completed.

Contents

1 — Purpose and scope	3
2 — Overall system architecture	4
3 — End-to-end analysis pipeline	5
4 — Software component diagram	6
5 — Scientific data flow	7
6 — C4 container view	9
7 — Generic framework vs FGFR2 specialization	11
8 — Single-species and multi-species behaviour	13
9 — Researcher interaction workflow	14
10 — LRZ cluster round trip	14
11 — InterProScan / pyTMHMM normalization	16
12 — Exon-domain boundary methodology	18
13 — Run-directory and artifact lifecycle	18
14 — Reproducibility and provenance	18
15 — Current limitations and planned extensions	19
16 — Building the Architecture Atlas	19
17 — Diagram legend and glossary	19
18 — Repository paths corresponding to the architecture	20
Appendix — Overview poster	21

1 — Purpose and scope

ExonDomainCompare is a comparative bioinformatics application that investigates **how protein-domain edges relate to coding-exon boundaries**, and whether that relationship is conserved across species. For a chosen gene and one or more vertebrate species it collects transcript and protein models, selects a primary model per species, maps exons onto protein coordinates, annotates domains and transmembrane topology on a compute cluster, and quantifies the relationship between coding-exon boundaries and representative domain edges. Results are explored interactively in a web application and exported as figures and tables.

This guide explains the architecture so a reader who has never seen the repository can understand what the project does, how a run progresses from gene input to scientific results, which software components are involved, how local and LRZ cluster processing interact, how single- and multi-species analyses use the same framework, and how the validated FGFR2 use case extends the generic framework.

The central architectural idea

There is exactly **one generic, annotation-aware exon–protein analysis framework**. Any gene and any number of species flow through the same shared modules. **FGFR2 is a specialization layered on top** of that framework: it adds IIIb/IIIc cassette interpretation, validated evidence and cassette-aware overlays, but it does **not** fork the pipeline. Generic genes (for example the FGFR1, TPM1 and TP53 core-only pilots) never receive IIIb/IIIc terminology; they receive generic isoform alignment, exploratory candidate evidence and generic boundary interpretation instead.

How to read the colours. Every diagram uses one shared semantic palette: User / frontend Local compute LRZ cluster Scientific data Annotation / boundary Generic mode Outputs FGFR2 specialization. The shared framework is always visually dominant; FGFR2 uses a distinct rose accent.

Arrow and shape vocabulary

- **Solid arrow** — primary data flow.
- **Dashed arrow** — optional or conditional flow (e.g. exploratory candidates, multi-species-only MSA).
- **Dotted line** — configuration or reference relationship (e.g. reading cached inputs).
- **Rose accent border / rose arrow** — FGFR2 specialization.
- **Cloud** — external system; **cylinder** — persistent store; **document** — data file; **person** — the researcher.

2 – Overall system architecture

The complete system at one glance, from researcher to interactive results.

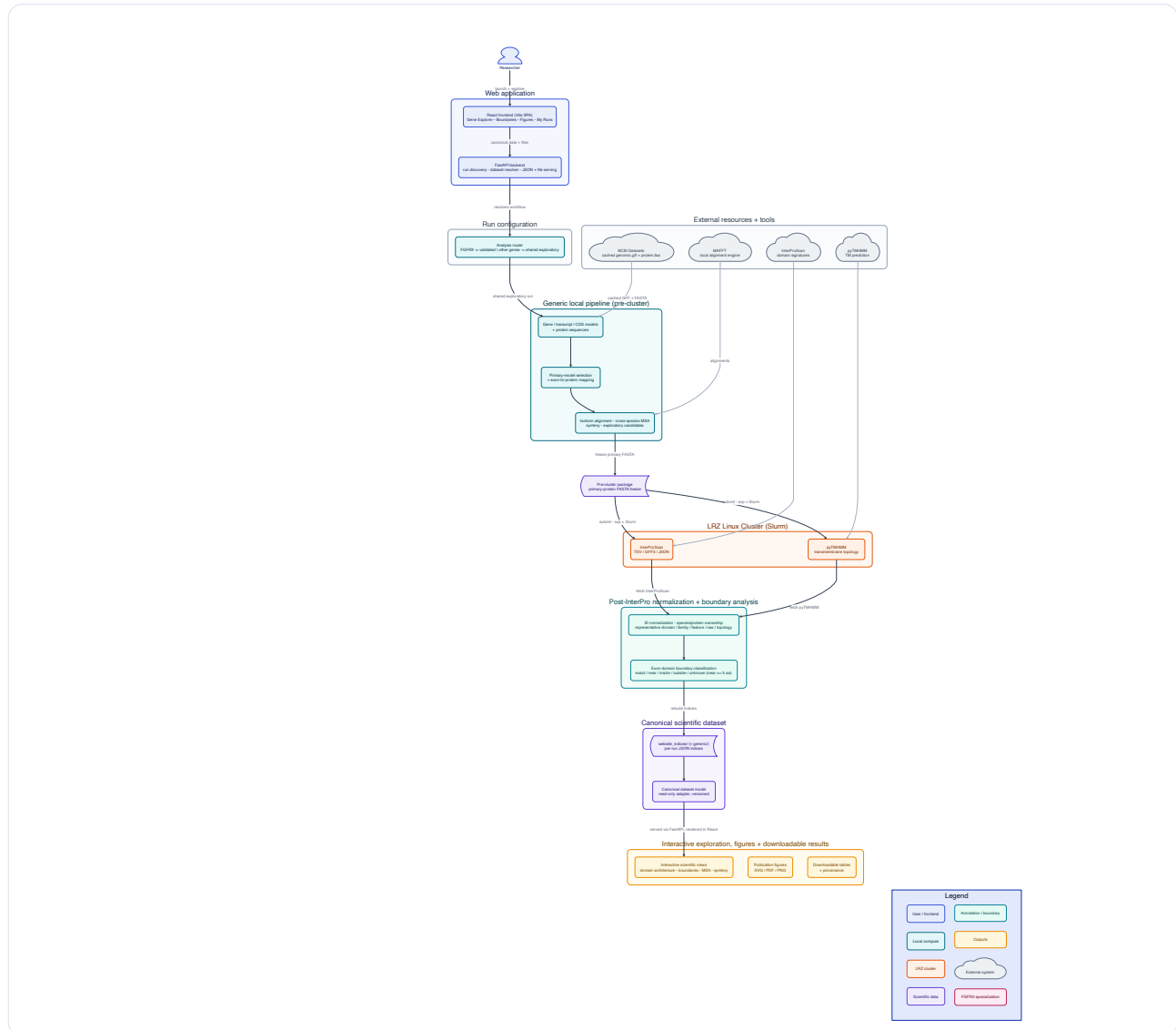


Figure 2. The value chain flows top-to-bottom: the researcher works in the React web application; the FastAPI backend resolves the workflow (FGFR2 → validated, other genes → shared exploratory); the generic local pipeline builds models, selects primaries, maps exons, aligns isoforms and computes synteny; the primary-protein FASTA is frozen and submitted to the LRZ cluster for InterProScan and pyTMHMM; results are fetched and normalized into layered annotations and boundary classifications; the canonical dataset is served back through the backend into interactive views, figures and downloads. FGFR2 reuses this identical spine and adds cassette-aware layers.

Repository. Frontend webapp/frontend/src/; backend webapp/backend/main.py; router scripts/framework/analysis_router.py; local pipeline scripts/framework/run_core_gene_analysis.py + scripts/generic_gene/; cluster scripts/interpro_cluster/; normalization scripts/framework/interpro_annotations.py; indices scripts/framework/build_core_gene_indices.py; canonical adapter webapp/backend/canonical_dataset.py.

3 — End-to-end analysis pipeline

The ordered stages of one run, grouped into local, cluster and post-cluster phases.

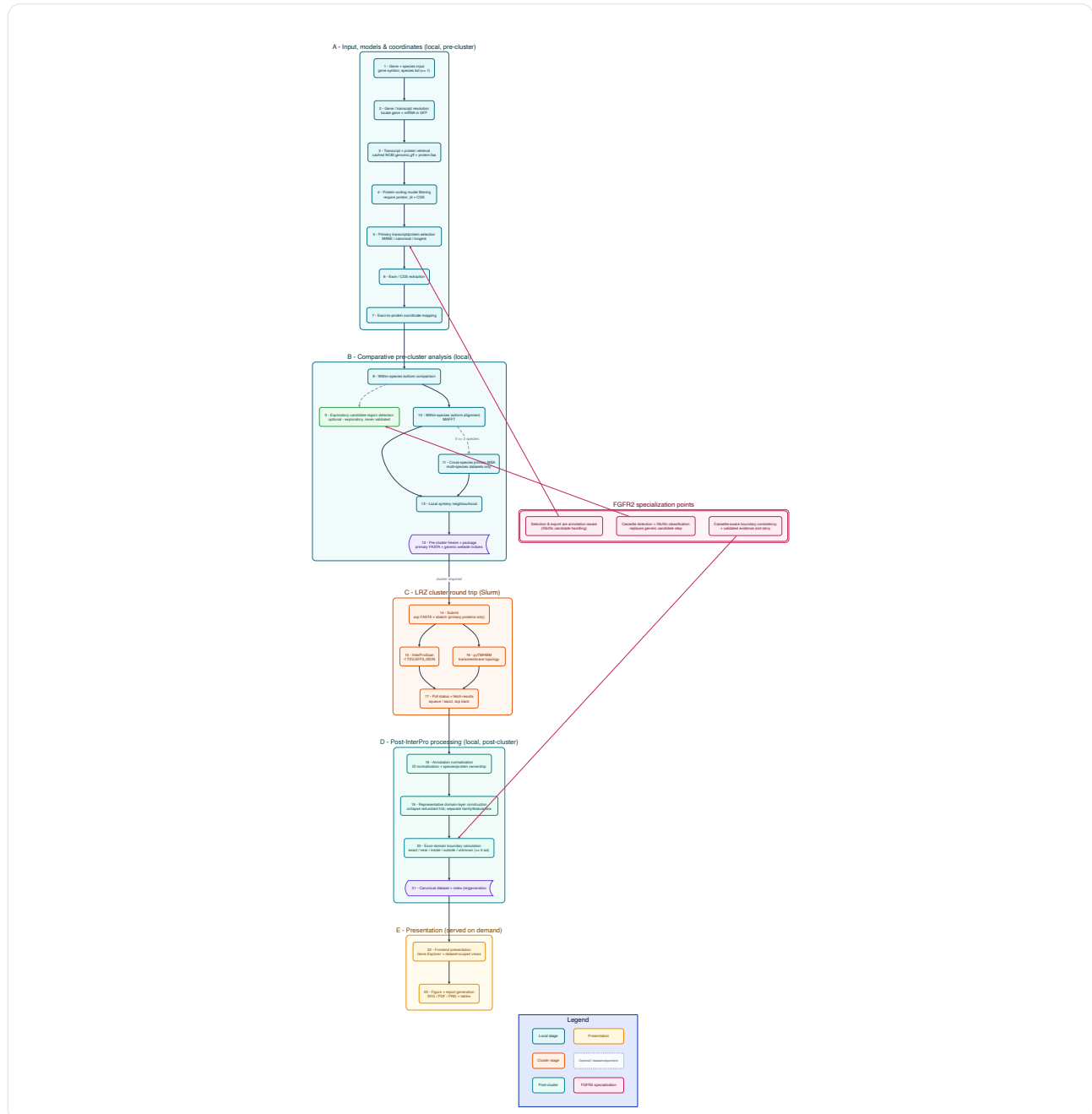


Figure 3. Phases A–B run locally (models, primary selection, exon mapping, isoform comparison/alignment, optional candidate detection, multi-species MSA, synteny and the pre-cluster freeze); phase C is the LRZ round trip; phase D normalizes annotations, builds the representative domain layer and classifies boundaries; phase E presents results. FGFR2 specialization points (rose) attach at selection/export, cassette classification and cassette-aware boundary consistency.

Repository. `run_core_gene_analysis.py` (`phase_create`, `parse_gene_models`, `_select_primary_pids`, `phase_post`); `run_generic_gene_analysis.py`; `interopro_cluster/run_cluster_roundtrip.py`.

4 – Software component diagram

Reusable components grouped by architectural layer, with dependencies and reuse.

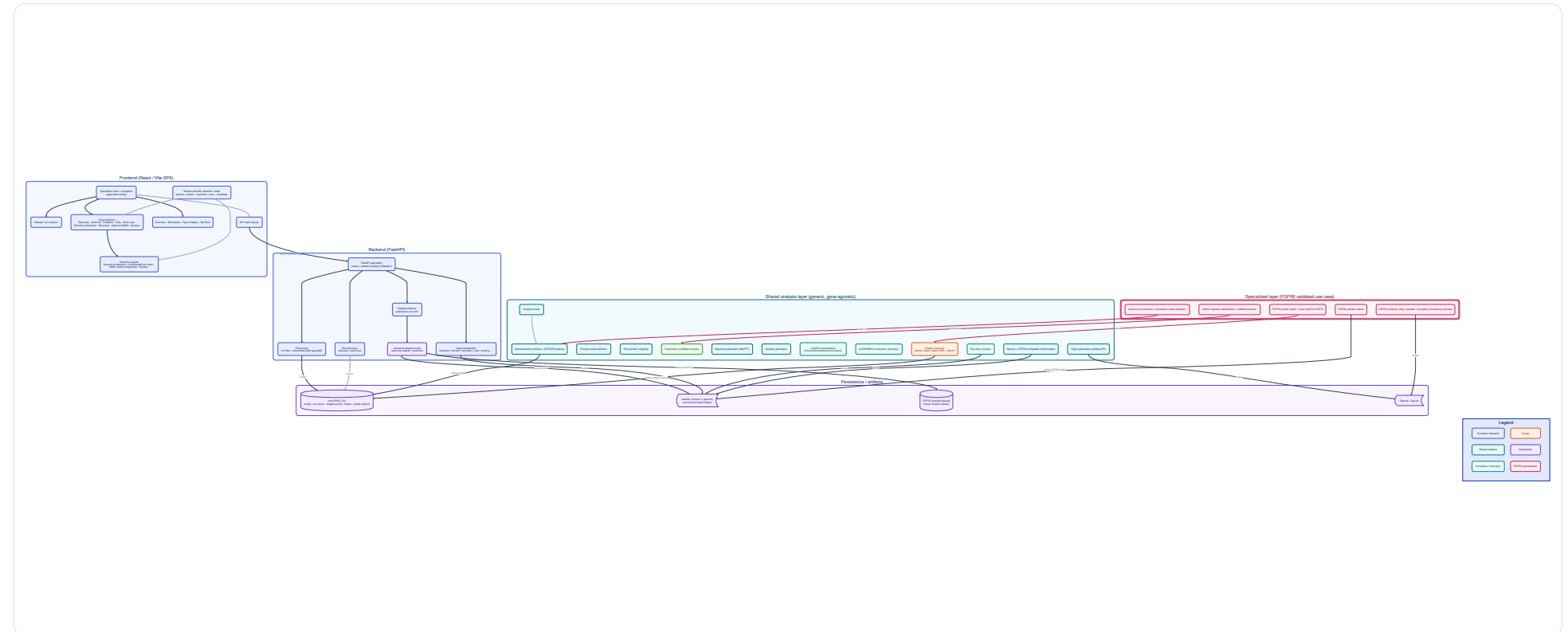


Figure 4. The frontend (React SPA) depends on the backend (FastAPI) over HTTP. The backend discovers runs, resolves datasets, exposes a read-only canonical dataset model and serves indices and files from run-directory storage. The shared analysis layer (generic, gene-agnostic) produces all run artifacts and indices. The specialized layer builds on the shared layer for FGFR2 and adds cassette classification, validated evidence and FGFR2 indices. Persistence is the per-run directory plus the frozen FGFR2 example dataset.

Repository. Frontend App.jsx, pages/, pages/viewers/, components/ScientificSelectionContext.jsx, api.js; backend main.py, canonical_dataset.py; shared scripts/framework/, scripts/generic_gene/, scripts/shared_gene_analysis/, scripts/plotting/; specialized top-level scripts/*fgfr2*, scripts/build_website_indices.py.

5 — Scientific data flow

How biological data objects are transformed, with their backing files.

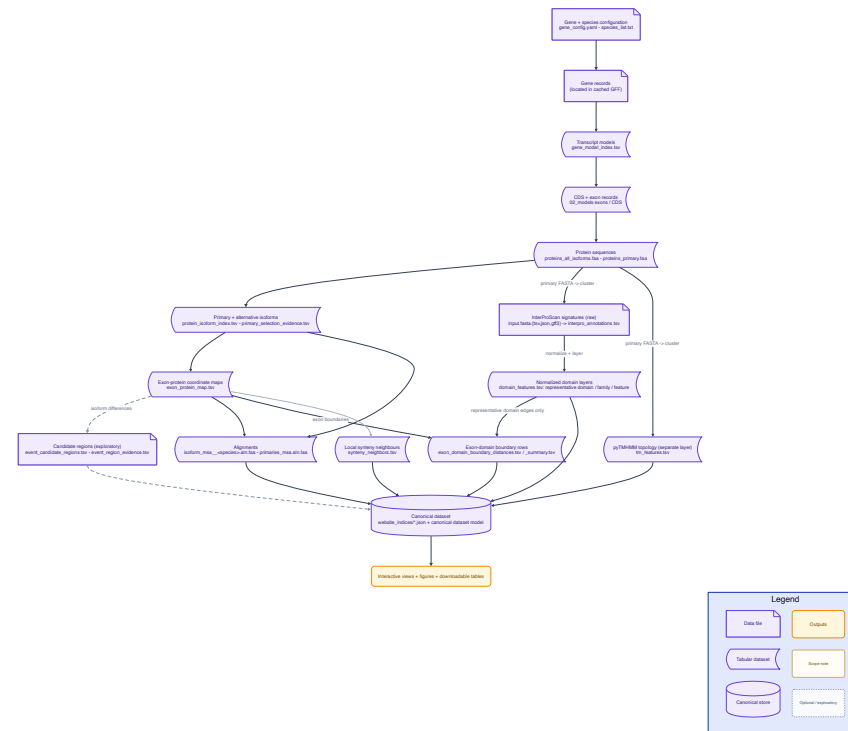


Figure 5. Configuration yields gene records, transcript models, CDS/exon records and protein sequences, from which primary and alternative isoforms and exon–protein coordinate maps are derived. Only **primary proteins** are submitted to the cluster; raw InterProScan signatures are preserved for provenance and normalized into layered annotations, while pyTMHMM topology is kept separate. Exon boundaries are combined with the **representative domain layer only** to produce boundary rows. Everything converges into the canonical dataset and then into interactive views, figures and tables.

Repository. `run_core_gene_analysis.py` writes `exon_protein_map.tsv`, `interpro_annotations.tsv` (raw), `domain_features.tsv` (normalized), `tm_features.tsv`, `exon_domain_boundary_distances.tsv`; `interpro_annotations.py` performs layer assignment.

6 – C4 container view

Containers, responsibilities and principal data exchanged (C4 adapted to scientific computing).

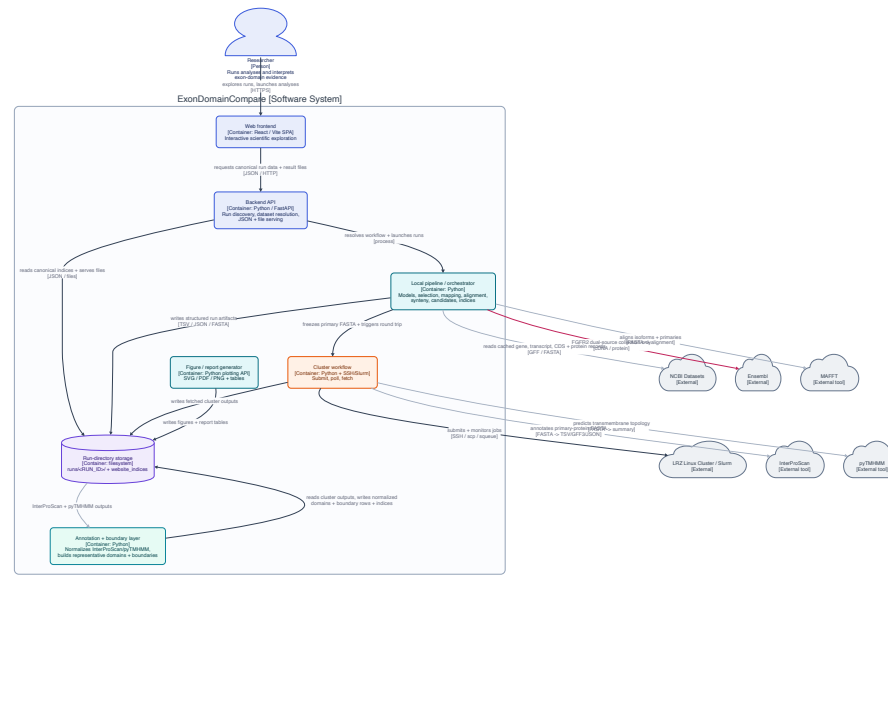


Figure 6. The researcher interacts with the web frontend, which requests canonical run data and files from the backend API. The backend reads canonical indices and launches runs through the local pipeline/orchestrator, which writes run artifacts and reads cached NCBI records and MAFFT alignments. The cluster workflow submits and monitors Slurm jobs and invokes InterProScan and pyTMHMM. The annotation + boundary layer reads cluster outputs and writes normalized domains, boundary rows and indices; the figure generator writes figures and tables. FGFR2 additionally uses Ensembl for dual-source collection.

Repository. Containers map to webapp/frontend/, webapp/backend/, scripts/framework/ + scripts/generic_gene/, scripts/interpro_cluster/, scripts/plotting/, and runs/<RUN_ID>/.
External: NCBI Datasets, Ensembl (FGFR2 only), LRZ Slurm, InterProScan, pyTMHMM, MAFFT.

7 — Generic framework vs FGFR2 specialization

The central methodological contribution: one shared foundation, two modes built on it.

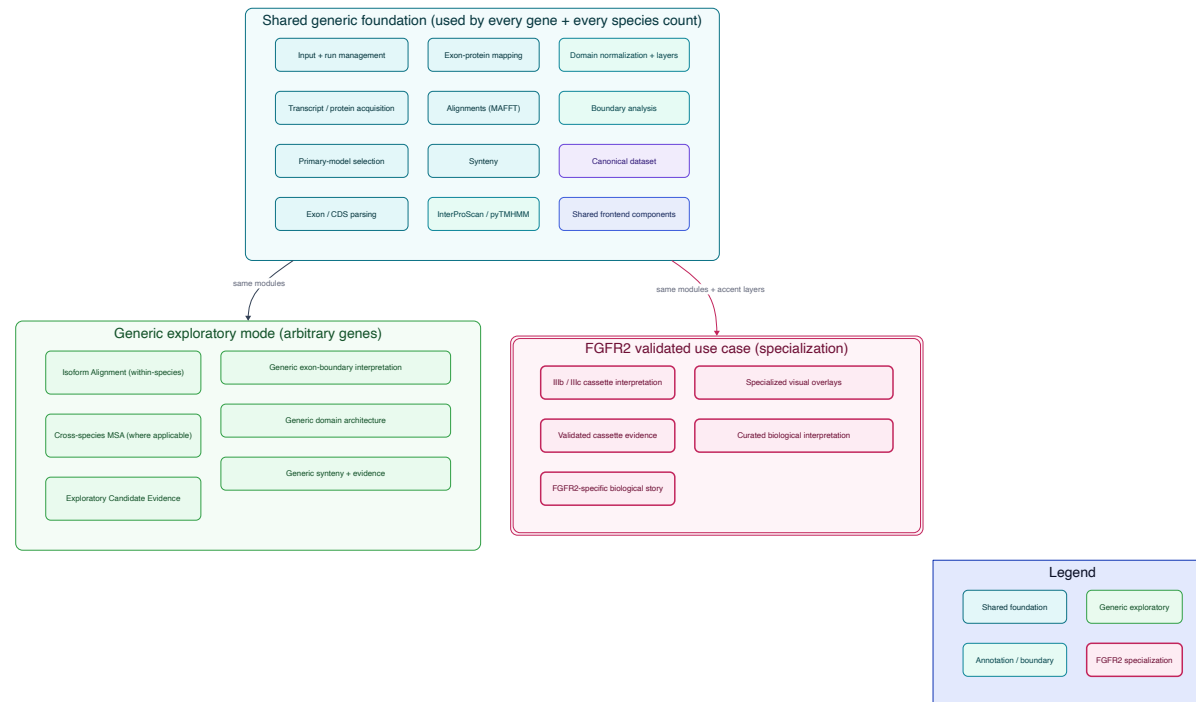


Figure 7. The shared generic foundation (input/run management, transcript/protein acquisition, primary-model selection, exon/CDS parsing, exon–protein mapping, alignments, synteny, InterProScan/pyTMHMM, domain normalization, boundary analysis, canonical dataset and shared frontend components) is used by every gene and every species count. The generic exploratory mode adds isoform alignment, exploratory candidate evidence and generic domain/boundary interpretation. The FGFR2 use case adds IIIb/IIIc interpretation, validated evidence, story and cassette-aware overlays — on the same modules, not a fork.

Repository. Routing `scripts/framework/analysis_router.py` (`FGFR2` → `validated_event_analysis`, else `shared_exploratory`); generic modes `scripts/generic_gene/`, `scripts/shared_gene_analysis/`; `FGFR2` config `configs/genes/FGFR2_IIb_IIIc.yaml`; pilots `configs/genes/drafts/{FGFR1,TPM1,TP53}_core_only_pilot.yaml`.

8 — Single-species and multi-species behaviour

Species count is a **dataset property, not a separate implementation**. The species list is always stored as a collection; a single-species run is simply a collection of length one. The identical pipeline runs for one, two or many species — there is no single-species code path to maintain.

Two semantics are worth stating precisely, because they are easy to confuse:

- **Within-species isoform alignment** compares the isoforms of one species to one another. It is always species-specific and is produced per selected species.
- **Cross-species MSA** compares **one primary protein per species**. It is only produced for multi-species datasets (two or more species with a primary protein).

In the frontend, the selected species drives every species-specific view; switching species changes the protein accession, domain annotations and boundary rows without leaking data across species.

Repository. Species collection in `scripts/framework/run_core_gene_analysis.py` (`species_ids` list, `create_new_run.normalize_species_token`); MSA logic in `scripts/generic_gene/build_generic_msa_index.py` (per-species isoform MSA vs multi-species `primaries_msa`); frontend species state in `webapp/frontend/src/pages/GeneExplorer.jsx` and `components/ScientificSelectionContext.jsx`.

9 — Researcher interaction workflow

A realistic analytical journey and how selections synchronize the views.

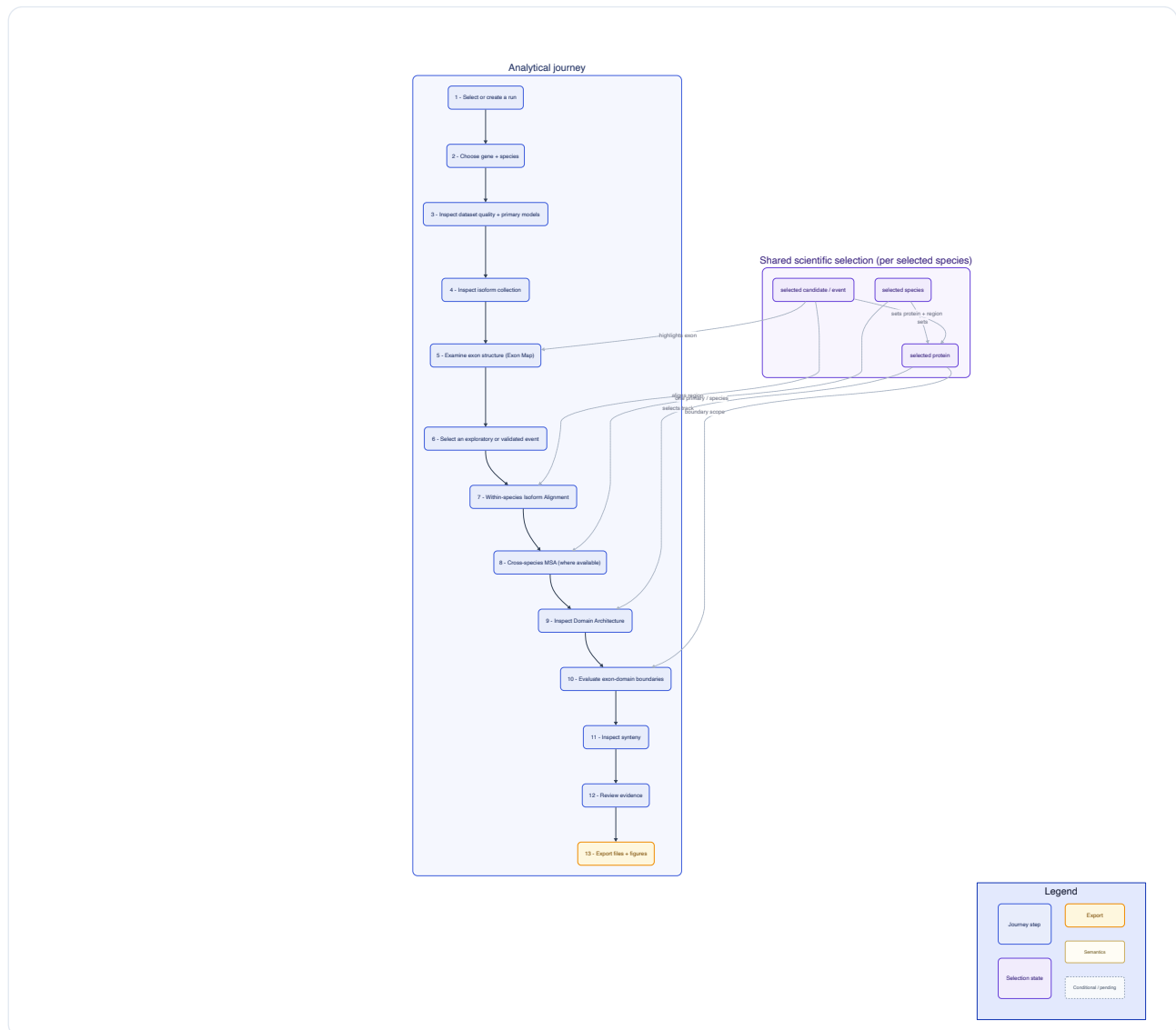


Figure 9. The researcher selects or creates a run, chooses gene and species, inspects dataset quality and primary models, examines the isoform collection and exon structure, selects an exploratory or validated event, and then inspects alignment, domain architecture, boundaries, synteny and evidence before exporting. A shared selection state (species, protein, candidate/event) synchronizes the coordinated views: selecting a candidate sets the protein and alignment region and highlights exons; selecting a species scopes the cross-species MSA to one primary per species.

Repository. `webapp/frontend/src/components/ScientificSelectionContext.jsx` (selection state); `pages/GeneExplorer.jsx`, `pages/viewers/CoordinateTrack.jsx`, `pages/viewers/DomainArchitecture.jsx`, `pages/viewers/MsaExplorer.jsx`, `pages/BoundaryPage.jsx`.

10 — LRZ cluster round trip

Domain and topology annotation runs on the LRZ Linux Cluster via Slurm. The round trip is orchestrated locally by `scripts/interpro_cluster/run_cluster_roundtrip.py` in four steps:

1. **Submit** — the frozen primary-protein FASTA (`13_final_pre_interpro_closure/freeze/final_pre_interpro_proteins_primary.faa`) is copied to the InterProScan and pyTMHMM input folders and transferred to the remote run directory; two Slurm jobs are submitted (InterProScan and pyTMHMM run **in parallel**).
2. **Check** — job state is polled with `squeue`, falling back to `sacct`, and written to `status.json`.
3. **Fetch** — remote outputs are discovered and copied back with `scp` into `results/14_interproscan/primary/output/` (`input.fasta`, `{tsv,gff3,json}`) and `results/15_exon_domain_boundary_post_interpro/pytmhmm_primary/output/` (`pytmhmm_summary_all.tsv`, per-protein `*.summary/*.annotation/*.plot`).
4. **Post** — post-cluster processing is triggered automatically (`run_core_gene_analysis.py --post` for generic runs; `run_post_interpro_for_run.py` for FGFR2).

Cluster annotation scope. Only **primary proteins** are submitted for InterProScan and pyTMHMM. Alternative isoforms are not annotated, so the frontend must never imply that unsubmitted isoforms have domain annotations.

Repository. `scripts/interpro_cluster/{run_cluster_roundtrip,submit_cluster_analysis,check_cluster_analysis,fetch_cluster_analysis,ssh_common}.py`. Remote host `LRZ_LOGIN_HOST`; SSH multiplexing via `ControlMaster`.

11 — InterProScan / pyTMHMM normalization

The post-cluster phase, with explicit annotation-layer discipline.

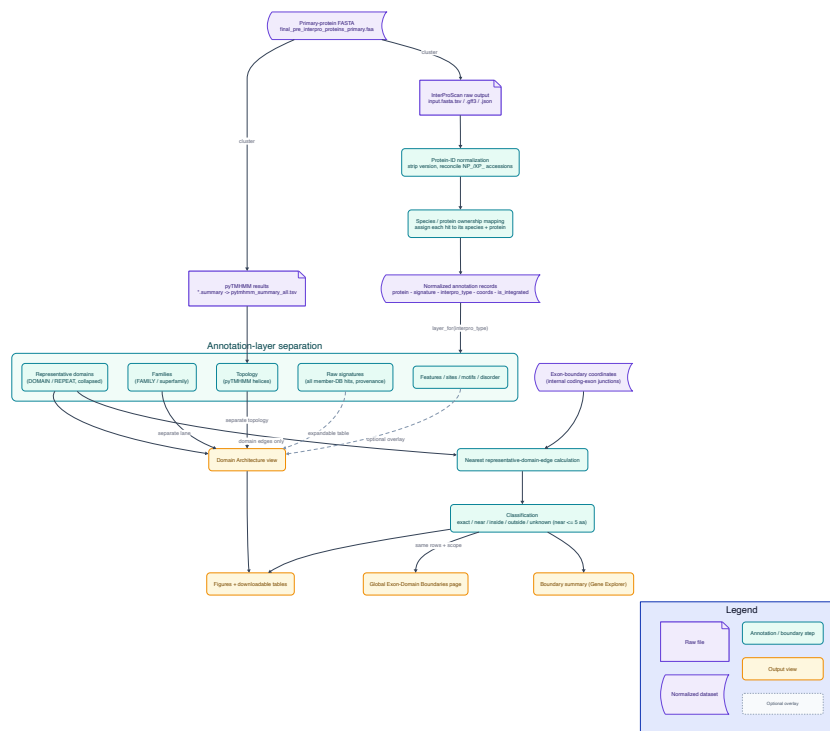


Figure 11. Raw InterProScan output and pyTMHMM results are parsed; protein IDs are normalized (version stripping, NP/XP reconciliation) and each hit is assigned to its species and protein. Normalized records are separated into layers: representative domains (DOMAIN/REPEAT, collapsed), families (FAMILY/superfamily), features (sites/motifs/disorder), raw signatures (all member-database hits, kept for provenance) and topology (pyTMHMM). Only the representative domain layer feeds boundary edges; families are a separate lane, features are optional overlays, raw hits are an expandable table and pyTMHMM is a separate topology layer.

Repository. scripts/framework/interpro_annotations.py (layer_for, parse_interproscan_json/tsv, representative_domains, family_annotations, feature_annotations);
run_core_gene_analysis.py phase_post; index assembly in build_core_gene_indices.py.

12 — Exon–domain boundary methodology

For each internal coding-exon boundary of a primary protein, the nearest edge of the **representative domain layer** is found and the absolute amino-acid distance is computed. Each boundary is classified as:

- `exact_edge` — the boundary coincides with a domain edge;
- `near_edge` — within the near-edge threshold (default **5 aa**);
- `inside_domain` — the boundary lies within a domain;
- `outside_domain` — the boundary lies between domains;
- `unknown` — no representative domain model is available (absence of a model is not evidence of an outside relationship).

Family annotations, disorder predictions, sites and motifs are **not** used as domain edges. Every boundary row retains species, transcript, protein, boundary position, nearest domain accession/name/type, nearest edge, absolute distance, classification and the domain layer used — so that the Gene Explorer boundary summary and the global Exon–Domain Boundaries page are computed from the same rows and the same scope.

```
Repository.      Classification      constants      NEAR_EDGE_THRESHOLD_AA      =      5      in      scripts/framework/
build_core_gene_indices.py;      boundary      computation      in      run_core_gene_analysis.py      (_classify_boundary,
phase_post); outputs exon_domain_boundary_distances.tsv, exon_domain_boundary_summary.tsv, website_indices/generic/
exon_domain_boundary_index.json.
```

13 — Run-directory and artifact lifecycle

Each run has its own identifier and an isolated directory `runs/<RUN_ID>/`. The directory follows canonical numbered stages so that pre-cluster and post-cluster artifacts are unambiguous:

```
runs/<RUN_ID>/
gene_config.yaml, species_list.txt, run_config.json, status.json, logs/
results/01_species_registry/ ... 02_models/ ... 06_coordinate_mapping/ ... 07_msa/ ... 08_syteny/
results/10_figures_pre_domain/ ... 13_final_pre_interpro_closure/freeze/ (cluster input)
results/14_interproscan/primary/{input,output,slurm,logs}/ (fetched)
results/15_exon_domain_boundary_post_interpro/pytmhmm_primary/{input,output,...}/
results/15_domain_architecture/ ... 16_final_analyses/
results/core_gene_analysis/ (canonical contract TSVs)
results/generic_gene_analysis/ (materialized generic products)
website_indices/ (+ generic/) (frontend JSON indices)
```

The frozen FGFR2 **example** dataset is served read-only from `results/final_30_until_interpro_prepare/13_final_pre_interpro_closure/website_indices/` and is never rewritten.

14 — Reproducibility and provenance

Runs are self-contained: inputs, intermediate results, cluster outputs, indices and reports live under a single run directory keyed by run ID. Raw InterProScan signatures are retained alongside the curated layers, so every displayed domain can be traced back to its member-database hits and coordinates. The canonical dataset model is a **read-only, versioned adapter** over the on-disk indices; it creates no new

biological data. This Architecture Atlas is itself reproducible — all diagrams and this guide are generated from editable sources by a single command (see §16).

15 — Current limitations and planned extensions

- **Primary-protein annotation only.** InterProScan/pyTMHMM currently annotate primary proteins; alternative isoforms are not annotated. current scope
- **Generic core uses cached NCBI models.** The generic core runner consumes cached NCBI Datasets GFF/FASTA; live acquisition and user-supplied inputs are partly planned. FGFR2 uses dual-source NCBI + Ensembl collection.
- **Domain/boundary views require the cluster.** Before InterProScan/pyTMHMM complete, Domain Architecture and Boundary views show a calm pending state rather than fabricated results.
- **Validated events are FGFR2-only.** Only FGFR2_IIIb_IIIc.yaml is a validated event configuration; FGFR1/TPM1/TP53 exist as core-only pilots.
- **Exploratory candidates are not validated.** Generic candidate regions are exploratory evidence and never asserted as validated events.

16 — Building the Architecture Atlas

All artifacts are reproducible from source:

```
# one-time toolchain (Homebrew):
brew install d2 librsvg typst

# build every SVG + PDF diagram and this guide:
python scripts/docs/build_architecture_atlas.py
# or: make architecture

# subsets:
python scripts/docs/build_architecture_atlas.py --diagrams
python scripts/docs/build_architecture_atlas.py --guide
python scripts/docs/build_architecture_atlas.py --check # verify tools
```

Diagram sources are D2 (docs/architecture/sources/*.d2, shared identity in _theme.d2); the guide is Typst (docs/architecture/guide/). Each diagram is kept as source, SVG (docs/architecture/svg/) and distribution PDF (docs/architecture/).

17 — Diagram legend and glossary

Legend

User / frontend	React SPA and FastAPI-facing components.
Local compute	Local pre-cluster pipeline stages.
LRZ cluster	Slurm jobs: InterProScan, pyTMHMM.
Scientific data	Data objects, files and canonical dataset.
Annotation / boundary	Normalization, layering and boundary analysis.
Generic mode	Generic exploratory analysis for arbitrary genes.
Outputs	Interactive views, figures, downloadable tables.
FGFR2 specialization	Cassette-aware validated FGFR2 layers.

Glossary

Primary model The representative transcript/protein selected per species (MANE/canonical/longest).

Representative domain layer Deduplicated, collapsed structural domains (DOMAIN/REPEAT) used for the default architecture and boundary edges.

Family layer FAMILY / homologous-superfamily entries, displayed separately, never counted as domain edges.

Feature layer Sites, motifs and disorder predictions; optional overlays, not domain edges.

Raw-signature layer All member-database hits, retained for provenance.

Topology layer pyTMHMM transmembrane helices; a separate layer, never an InterPro domain.

Canonical dataset The versioned, read-only frontend model assembled over per-run JSON indices.

Pre-cluster / post-cluster Locally computable stages vs. stages requiring real InterProScan/pyTMHMM results.

Core-only pilot A generic run for a gene with no validated event configuration (e.g. FGFR1, TPM1, TP53).

18 — Repository paths corresponding to the architecture

```
Frontend — webapp/frontend/src/ (App.jsx, pages/, pages/viewers/, components/ScientificSelectionContext.jsx, api.js)
Backend — webapp/backend/main.py, webapp/backend/canonical_dataset.py
Analysis router — scripts/framework/analysis_router.py
Generic local pipeline — scripts/framework/run_core_gene_analysis.py, scripts/generic_gene/run_generic_gene_analysis.py + modules
Shared / FGFR2-compatible indices — scripts/shared_gene_analysis/, scripts/framework/build_core_gene_indices.py, scripts/generic_gene/build_generic_website_indices.py
InterPro normalization — scripts/framework/interpro_annotations.py
Cluster round trip — scripts/interpro_cluster/
Plotting — scripts/plotting/
FGFR2 specialization — scripts/*fgfr2*, scripts/build_website_indices.py, configs/genes/FGFR2_IIIb_IIIc.yaml
Contracts / docs — docs/core_gene_analysis_contract.md, docs/event_detector_contract.md, docs/plotting_api.md
Tests — tests/test_interpro_annotation_layers.py, tests/test_post_interpro_generic.py, tests/test_canonical_dataset.py
Runs — runs/<RUN_ID>; frozen example under results/final_30_until_interpro_prepare/13_final_pre_interpro_closure/
```

A component-by-component mapping is maintained in docs/architecture/architecture_inventory.md.

Appendix — Overview poster

A single-sheet summary for presentations and thesis appendix use (also available as [docs/architecture/09_exondomaincompare_architecture_poster.pdf](#)).

